# Comparação: Space Invaders do Claude e do Gemini (primeira versão)

Oct 6, 2026 · @André

## Resumo

As duas primeiras versões abrem e dá para jogar, mas nenhuma estava completa. A versão do Claude trouxe mais estrutura de POO e mais regras de jogo: vidas, pontos, inimigos que atiram, vitória e derrota. A do Gemini é mais curta e simples: os inimigos não atiram, o jogador não pode perder e destruir todos os inimigos não faz nada. Nenhuma das duas tinha tela de início, tela de fim ou botão de reiniciar.

A comparação usa a primeira versão de cada jogo, antes de qualquer alteração do grupo. Os pontos do jogo do Gemini foram conferidos rodando o código no navegador (Chromium) e lendo o arquivo. Os pontos do Claude vieram da leitura do código e dos prints da primeira versão.

## Como cada IA foi usada

O Claude recebeu o enunciado e foi conduzido em três prompts, até gerar o projeto completo. O Gemini recebeu apenas o print do enunciado e o pedido de um jogo no estilo Space Invaders, e respondeu com o código em uma única resposta.

| Item | Claude | Gemini |
|---|---|---|
| Entrada | Enunciado da NI1 em PDF e perguntas curtas | Print do enunciado da NI1 e o pedido de um jogo estilo Space Invaders |
| Prompts até ter o jogo | 3 | 1 |
| Prompt 1 | "Quais ideias vc acha bom para a criação desse projeto? O professor falou que pode ser bem simples" | (pedido descrito pelo André, sem o texto exato) |
| Prompt 2 | "Vamos com o space invaders, mas só para confirmar, o vs code daria conta normal?" | não houve |
| Prompt 3 | "Pode montar" | não houve |

| Item | Claude | Gemini |
|---|---|---|
| Primeira resposta | Cinco ideias de jogo, com recomendação de Space Invaders ou RPG por turnos | Código base do jogo, com explicação de onde aparece cada conceito de POO |
| Formato da entrega | 10 arquivos (HTML, CSS e 8 classes em JavaScript) | 3 arquivos no projeto recebido ( `index.html` , `style.css` e `script.js` ) |
| Idioma da resposta | Português do Brasil | Português de Portugal ("ecrã", "ficheiro", "podes") |

Na resposta do Gemini, ele montou a tela vertical de 360 x 640, listou como classes, herança e polimorfismo aparecem no código e lembrou o grupo de tirar prints e documentar o prompt, porque o enunciado exige a comparação entre IAs. O print da resposta está na seção de evidências.

## Estrutura do código

O Claude dividiu o jogo em oito classes, uma por arquivo. O Gemini concentrou quatro classes e a lógica do jogo em um único `script.js` .

| Item | Claude (1ª versão) | Gemini (1ª versão) |
|---|---|---|
| Arquivos | 10: `index.html` , `style.css` e 8 arquivos `.js` | 3: `index.html` , `style.css` e `script.js` |
| Linhas de JavaScript | 302 | 166 |
| Linhas com comentário ( `//` ) | 54 | 31 |
| Classes | 8 | 4 |
| Superclasse | `Entidade` | `EntidadeJogo` |
| Subclasses | `Jogador` , `Projetil` e `Inimigo` ; de `Inimigo` herdam `InimigoBasico` , `InimigoRapido` e `InimigoAtirador` | `Jogador` , `Inimigo` e `Projetil` |
| Níveis de herança | 2 | 1 |
| Tipos de inimigo | 3 | 1 |

| Item | Claude (1ª versão) | Gemini (1ª versão) |
|---|---|---|
| Quantidade de inimigos | 32 (4 linhas de 8) | 28 (4 linhas de 7) |
| Tamanho da tela | 800 x 600 (horizontal) | 360 x 640 (vertical) |
| Quem controla o jogo | Classe `Jogo`, com loop, colisões e fim de jogo | Variáveis e funções soltas no `script.js` (`inicializarInimigos` e `loopJogo`) |

## O que cada jogo faz ao rodar

O Claude cumpre nove dos onze itens, dois deles em parte, e o Gemini cumpre três. O que falta nas duas versões é a tela de início, a tela de fim e o botão de reiniciar.

| Funcionalidade | Claude (1ª versão) | Gemini (1ª versão) |
|---|---|---|
| Tela de início | Não: o jogo começa sozinho ao abrir a página | Não: o jogo começa sozinho ao abrir a página |
| Nave se move e atira | Sim, com setas e espaço; há intervalo de 15 quadros entre os tiros | Sim, com setas e espaço; não há intervalo, e cada repetição da tecla gera um tiro |
| Inimigos andam em bloco e descem | Sim | Sim |
| Tiro do jogador destrói inimigo | Sim | Sim |
| Inimigos atiram na nave | Sim, mas só o tipo Atirador (0,2% de chance por quadro) | Não: o código não tem nenhum tiro de inimigo |
| Vidas | Sim, 3 vidas | Não |
| Pontuação | Sim, 10, 20 ou 30 pontos conforme o tipo | Não |
| Derrota | Sim: sem vidas ou quando os invasores chegam à altura da nave | Não: os inimigos atravessam a nave, saem pela parte de baixo da tela e continuam descendo |
| Vitória | Sim: mensagem "Você venceu! Parabéns." | Não: ao destruir todos, fica só a nave numa tela preta |

| Funcionalidade | Claude (1ª versão) | Gemini (1ª versão) |
|---|---|---|
| Tela de fim e botão de reiniciar | Não: só uma mensagem de texto abaixo do jogo, e é preciso recarregar a página | Não: não existe fim de jogo |
| Colisão entre inimigo e nave | Por chegada à altura da nave | Não existe |

No teste do Gemini, depois que os inimigos chegaram à altura da nave, a posição vertical do primeiro deles passou de 585 para mais de 1000 em cerca de 25 segundos, com a tela de 640 de altura. Eles nunca voltaram para o topo.

## Problemas encontrados

A versão do Claude teve dois problemas de usabilidade. A do Gemini teve sete, quatro deles de regras do jogo que simplesmente não existiam.

| Versão | Problema | Causa no código |
|---|---|---|
| Claude | O jogo inicia sozinho ao abrir a página | A classe `Jogo` cria os objetos e chama `requestAnimationFrame` direto no construtor |
| Claude | Depois de vencer ou perder, só aparece uma mensagem de texto e é preciso recarregar a página | O método `_finalizar()` só escreve a mensagem e para o loop, sem como recomeçar |
| Gemini | Os inimigos não atiram | O código não tem nenhum tiro de inimigo; `Projetil` só sobe |
| Gemini | Não há derrota: os inimigos atravessam a nave e saem pela parte de baixo da tela | Nada compara a posição dos inimigos com a da nave, e a posição vertical deles aumenta sem limite |
| Gemini | Não há vitória: ao destruir todos os inimigos nada acontece | O loop não verifica se a lista de inimigos esvaziou |
| Gemini | Sem pontuação e sem vidas | Esses dados não existem no código |
| Gemini | Sem tela de início, de fim e botão de reiniciar | `loopJogo()` é chamado no fim do arquivo e não há outro estado do jogo |
| Gemini | Tiro sem intervalo | O disparo é criado a cada evento `keydown` do espaço, sem controle de tempo |

| Versão | Problema | Causa no código |
|---|---|---|
| Gemini | Risco de colisão falhar: pode pular ou remover o inimigo errado | O código usa `splice` dentro de um `forEach`, o que muda a lista durante a repetição. Identificado pela leitura do código, não reproduzido no teste |

Os dois problemas da versão do Claude foram corrigidos depois pelo André, com a tela de início e a tela de fim com botão de jogar novamente.

## Requisitos de POO do enunciado

As duas versões atendem ao mínimo pedido pelo enunciado, mas o Claude mostra os conceitos com mais clareza, principalmente o polimorfismo.

| Requisito | Claude (1ª versão) | Gemini (1ª versão) |
|---|---|---|
| Classes e objetos | 8 classes; o `Jogo` cria a nave, 32 inimigos e os projéteis | 4 classes; o script cria a nave, 28 inimigos e os projéteis |
| Herança | Dois níveis: `Entidade`, depois `Inimigo` e, a partir dele, três tipos de inimigo | Um nível: `EntidadeJogo` e suas três subclasses |
| Polimorfismo | `mover()` e `tentarAtirar()` são sobrescritos; o `Jogo` chama os mesmos métodos em inimigos de três tipos guardados na mesma lista | `atualizar()` é sobrescrito em `Jogador`, `Inimigo` e `Projetil`; não há lista com tipos diferentes de inimigo, então o ganho do polimorfismo é menor |
| Código organizado | Um arquivo por classe | Tudo em um único `script.js` |
| Código comentado | Comentários explicando onde há herança e polimorfismo | Comentários com rótulos como "Herança" e "Polimorfismo" nas próprias classes |
| Nomenclatura | Nomes em português, classes com inicial maiúscula | Nomes em português, classes com inicial maiúscula |

## Conclusão

A primeira versão do Claude ficou melhor como base do trabalho: tem mais regras de jogo, três tipos de inimigo e uma estrutura que mostra herança e polimorfismo com clareza. A do Gemini foi a mais rápida de obter, com um único prompt, e é mais fácil de ler por estar em um arquivo só, mas entregou um jogo que não termina: não dá para perder, ganhar nem reiniciar.

O ponto comum foi a falta de tela de início e de fim. No jogo do Claude, isso foi corrigido pelo André e documentado no documento principal do grupo. O jogo do Gemini foi mantido como está, para servir de comparação.

Cada IA também deixou uma lição para o grupo. Uma resposta rápida e curta, como a do Gemini, pode parecer completa e esconder regras que faltam, então o jogo precisa ser jogado até o fim. Já uma resposta mais completa, como a do Claude, ainda deixa lacunas de usabilidade que só aparecem ao testar no navegador.

## Evidências

**Conversa com o Gemini.** Parte da primeira resposta dele, com a explicação de classes, herança e polimorfismo:

Excelente ideia para o projeto da FECAP. Para ajudar na avaliação NI1 detalhada no ficheiro `image_cb2f39.png`, desenvolvi uma base funcional do jogo em JavaScript usando a tag `<canvas>`. PNG + 1

Configurei o ecrã do jogo com a proporção de 9:16 (resolução de 360x640), criando um formato vertical perfeito para o estilo arcada. Como o projeto exige a formação de grupos de 3 a 5 membros, sem exceção, podes partilhar este código inicial com o Clayton Santos Ferreira e os restantes colegas de equipa para que comecem a analisar a lógica, visto que todos terão de estar presentes na apresentação para explicar a sua contribuição e demonstrar conhecimento sobre o código. PNG + 1

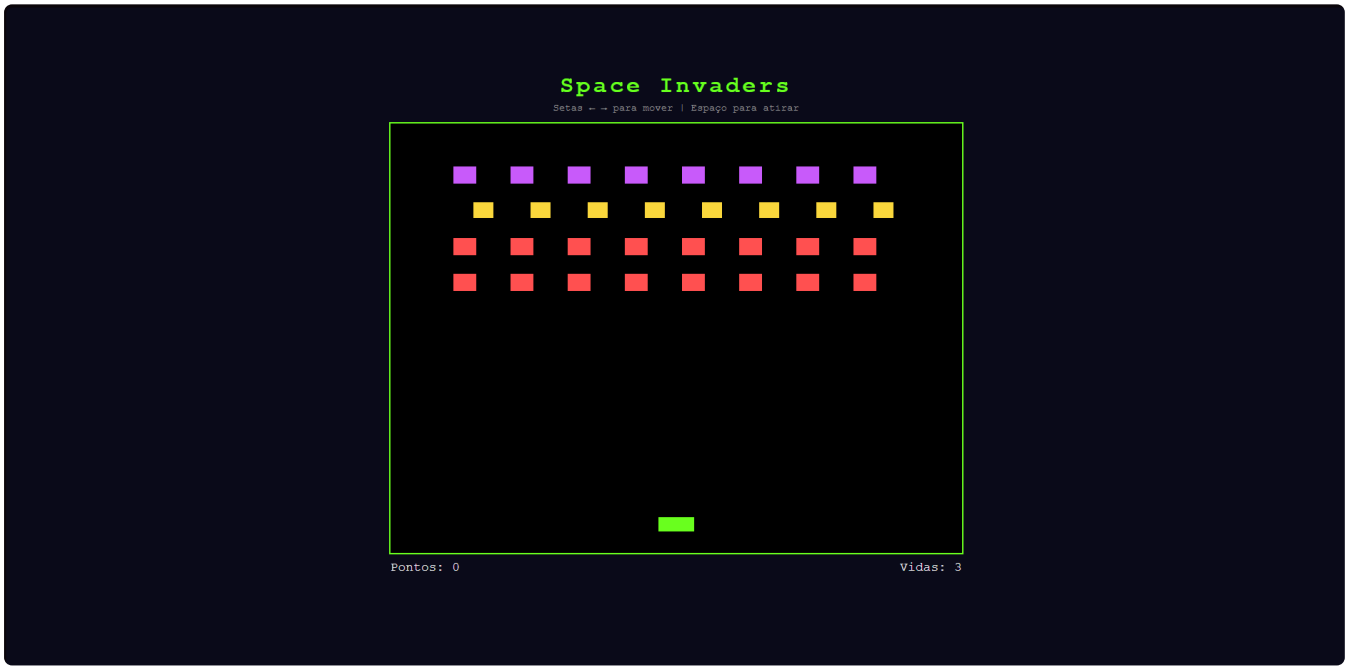
O código fonte abaixo foi construído para estar organizado e comentado com boas práticas de nomenclatura e indentação. Além disso, cobre os requisitos de Programação Orientada a Objetos exigidos: PNG + 1

- **Classes e Objetos:** Todas as entidades visuais são instanciadas a partir de classes. PNG
- **Herança:** As classes `Jogador`, `Inimigo` e `Projetil` herdam propriedades da superclasse base `EntidadeJogo`. PNG
- **Polimorfismo:** O método `atualizar()` existe na classe base, mas sofre sobrescrita (override) em cada subclasse para executar ações diferentes (o jogador move-se com as setas, o inimigo move-se de forma automática e o projétil sobe no ecrã). PNG

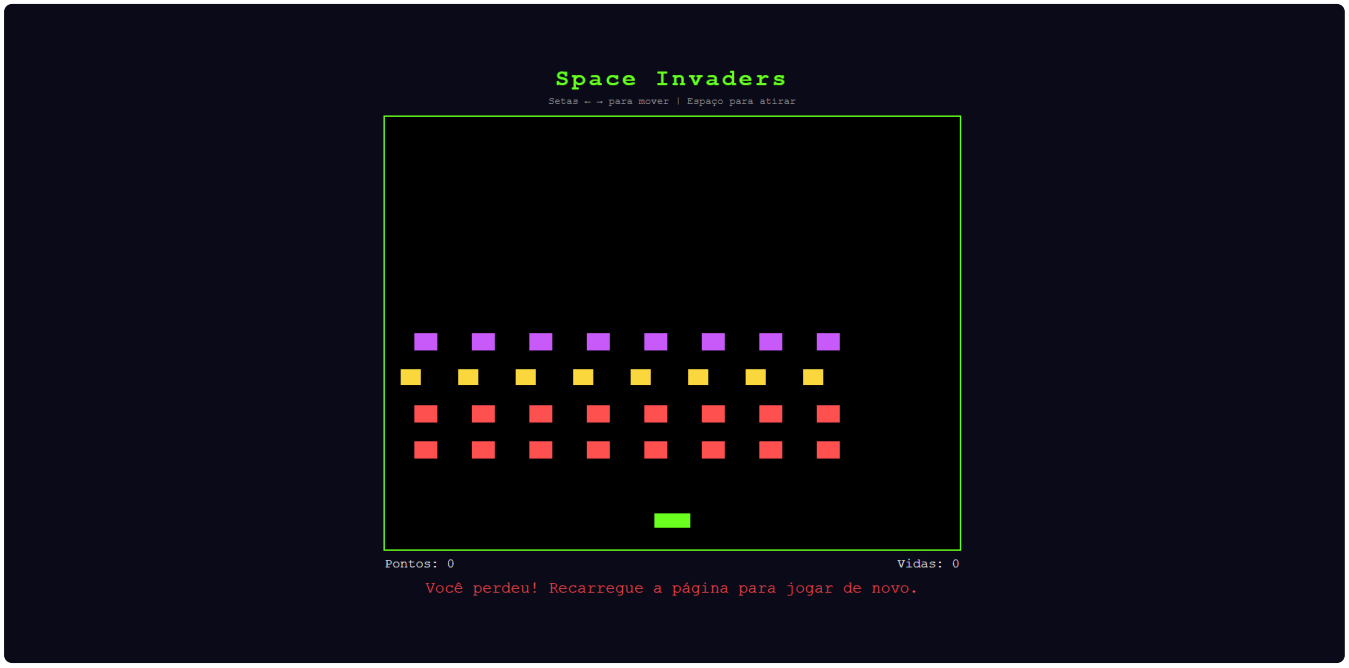
Lembra-te de tirar *prints* e documentar este *prompt* e o código gerado, pois o progresso do trabalho com o uso de ferramentas de IA deve ser bem documentado e comparado. PNG

Aqui tens o código base (podes guardar como um ficheiro `index.html` e abrir no navegador):

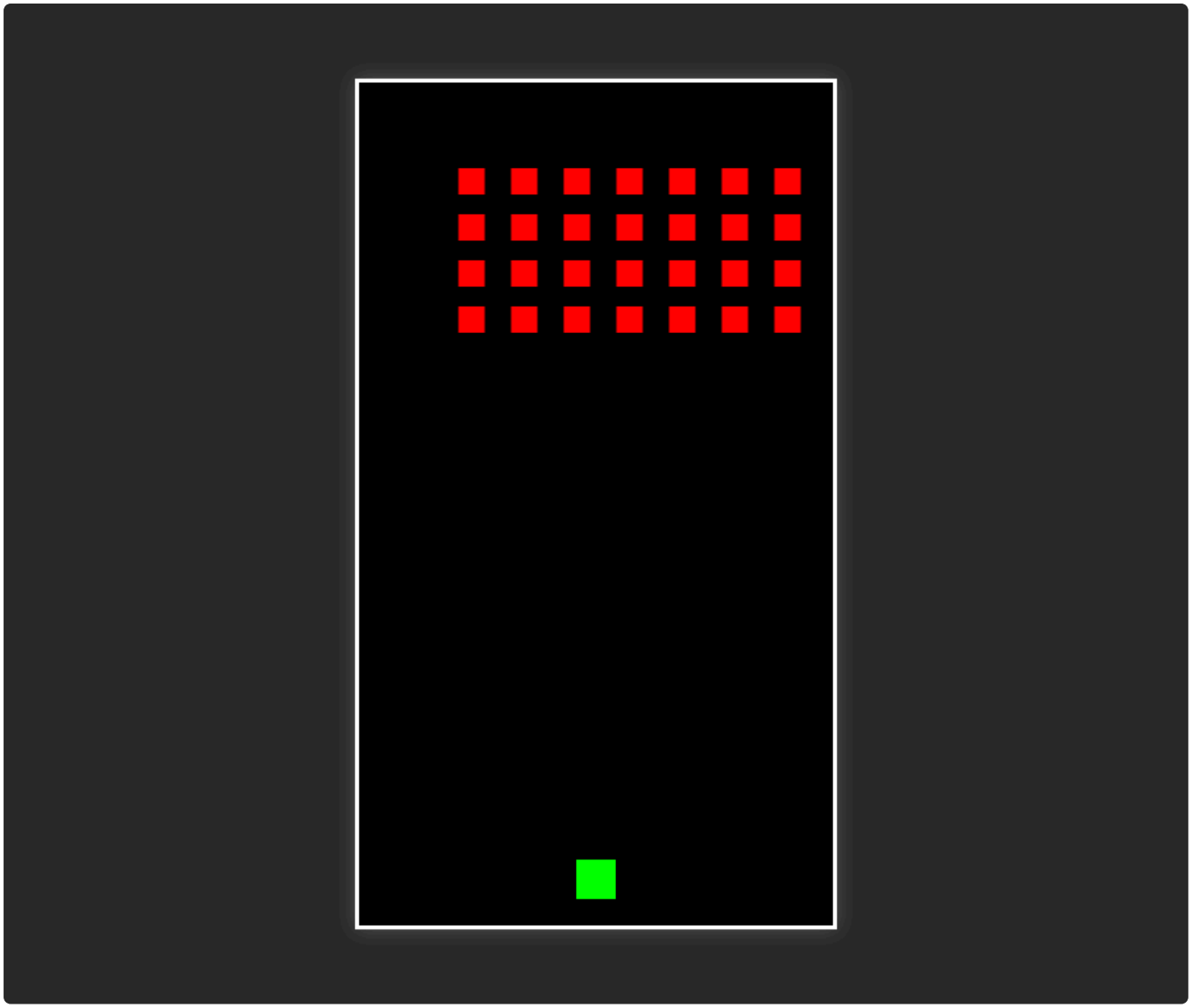
**Primeira versão do Claude.** O jogo já aparece em andamento ao abrir a página, sem tela de início:



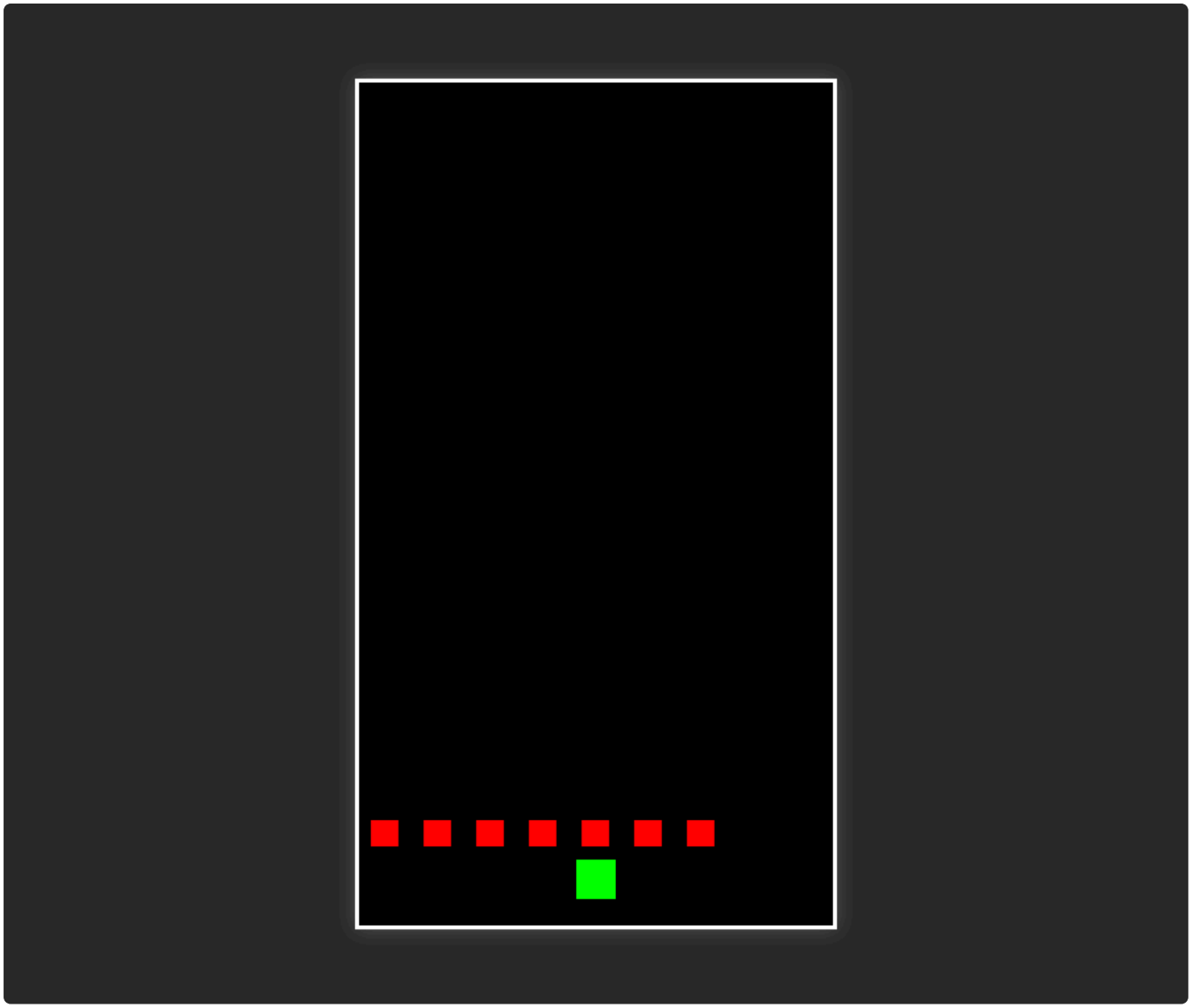
Ao perder, aparece só uma mensagem de texto pedindo para recarregar a página:




**Primeira versão do Gemini.** Tela vertical de 360 x 640, com 28 inimigos, que já estão em movimento ao abrir a página:



Quando os inimigos chegam à altura da nave, nada acontece: eles passam por ela e seguem descendo:



Depois que os inimigos saem pela parte de baixo, ou quando todos são destruídos, a tela fica igual: só a nave numa tela preta, sem mensagem de vitória nem de fim de jogo:

